

Desenvolvimento de Sensor Virtual para Estimativa de Torque em Motores de Indução utilizando Machine Learning

Diego Henrique de Araujo

Junho de 2026

Resumo

O monitoramento contínuo do torque em motores de indução trifásicos é essencial para a manutenção preditiva e a otimização de processos industriais. No entanto, a instalação de torquímetros físicos apresenta desafios significativos de custo, calibração e invasividade mecânica. Este trabalho apresenta o desenvolvimento de um sensor virtual para a estimativa de torque, utilizando sinais elétricos facilmente obtidos em inversores de frequência (VFD). O estudo emprega dados experimentais de alta fidelidade e compara o desempenho de duas abordagens clássicas de Aprendizado de Máquina: Regressão Linear e Random Forest. A metodologia inclui a aplicação da Transformada de Clarke para redução de dimensionalidade e pré-processamento de sinais. Os resultados demonstram que o modelo Random Forest captura com eficácia as dinâmicas não lineares do sistema eletromecânico, alcançando um coeficiente de determinação (R^2) superior a 0,99, enquanto a Regressão Linear apresenta desempenho insatisfatório devido à complexidade não linear inerente ao problema. A solução proposta viabiliza um monitoramento robusto e em tempo real, dispensando o uso de transdutores físicos.

Palavras-chave: Sensor Virtual, Motor de Indução, Machine Learning, Regressão Linear, Random Forest, Transformada de Clarke.

Sumário

1	Introdução	3
2	Fundamentação Física e Matemática	3
2.1	Modelo Eletromagnético do Motor de Indução	3
2.2	Transformada de Clarke	4
3	Metodologia	4
3.1	Base de Dados Experimental	4
3.2	Processamento e Engenharia de Dados	4
3.3	Modelos de Aprendizado de Máquina Avaliados	5
4	Resultados e Discussão	5
4.1	Desempenho Comparativo	5
4.2	Análise dos Resultados	6
5	Conclusão	7
A	Código de Reprodutibilidade	9

1 Introdução

O avanço da Indústria 4.0 tem impulsionado a adoção de técnicas avançadas de monitoramento de condição em equipamentos rotativos. Motores de indução trifásicos são os principais conversores de energia eletromecânica no setor industrial, e a estimativa precisa de seu torque mecânico é fundamental para programas de manutenção preditiva e controle de processos [2].

Tradicionalmente, a medição de torque exige a instalação de torquímetros de precisão no eixo do motor. Essa abordagem apresenta limitações severas: alto custo de aquisição, necessidade de parada operacional para instalação, desgaste mecânico e exigência de calibração periódica constante.

Como alternativa, os sensores virtuais (ou *soft sensors*) utilizam algoritmos matemáticos e de Aprendizado de Máquina para inferir variáveis complexas a partir de sinais de fácil medição. Em sistemas acionados por Inversores de Frequência (VFD – *Variable Frequency Drives*), correntes e tensões elétricas já são monitoradas em tempo real para fins de controle.

Este trabalho propõe o desenvolvimento de um sensor virtual de torque utilizando duas técnicas de Aprendizado de Máquina amplamente estudadas: a Regressão Linear e o algoritmo Random Forest. O objetivo principal é avaliar a capacidade dessas ferramentas em mapear a relação não linear entre as correntes estatóricas transformadas e o torque mecânico, utilizando uma base de dados experimental rigorosa.

2 Fundamentação Física e Matemática

2.1 Modelo Eletromagnético do Motor de Indução

A estimativa indireta de torque baseia-se na interação fundamental entre o fluxo magnético e as correntes que circulam no estator do motor. O torque eletromagnético (T_e) desenvolvido por um motor de indução pode ser expresso no referencial estacionário bifásico ($\alpha\beta$) pela seguinte equação [3]:

$$T_e = \frac{3}{2} P (\psi_{s\alpha} i_{s\beta} - \psi_{s\beta} i_{s\alpha}) \quad (1)$$

onde P é o número de pares de polos, $\psi_{s\alpha}$ e $\psi_{s\beta}$ são os componentes do fluxo estatórico, e $i_{s\alpha}$ e $i_{s\beta}$ são as correntes estatóricas no referencial estacionário. Esta relação demonstra que o torque depende do produto cruzado entre fluxos e correntes, evidenciando a natureza não linear do sistema eletromecânico.

2.2 Transformada de Clarke

A aquisição direta dos sinais em um motor trifásico fornece as correntes instantâneas das três fases (i_a, i_b, i_c). Para simplificar a análise matemática e reduzir a correlação entre as variáveis de entrada (uma vez que $i_a + i_b + i_c = 0$ em sistemas equilibrados), aplica-se a Transformada de Clarke.

Esta transformação matemática converte o sistema trifásico (abc) para um sistema ortogonal estacionário bifásico ($\alpha\beta$), reduzindo a dimensionalidade do problema de três para duas variáveis fundamentais, sem perda de informação. A transformação é dada por:

$$\begin{bmatrix} i_\alpha \\ i_\beta \end{bmatrix} = \frac{2}{3} \begin{bmatrix} 1 & -1/2 & -1/2 \\ 0 & \sqrt{3}/2 & -\sqrt{3}/2 \end{bmatrix} \begin{bmatrix} i_a \\ i_b \\ i_c \end{bmatrix} \quad (2)$$

Ao alimentar os algoritmos de Machine Learning com as correntes i_α e i_β , o modelo depara-se com *features* ortogonais, o que melhora significativamente a taxa de convergência e a estabilidade numérica durante a fase de treinamento.

3 Metodologia

3.1 Base de Dados Experimental

Para garantir a validade industrial da solução proposta, este trabalho não utiliza dados simulados. Emprega-se uma base de dados experimental oriunda de bancada de testes da Paderborn University, disponibilizada publicamente via plataforma Kaggle [4].

O conjunto de dados original consiste em aproximadamente 2,47 GB de medições contínuas em alta frequência, capturando o comportamento do motor em regimes transientes e permanentes, sob diferentes níveis de carga mecânica e frequência de acionamento. As variáveis brutas disponíveis incluem as correntes das três fases, a velocidade de rotação do eixo, a tensão no barramento DC do inversor e o torque mecânico real (variável alvo).

3.2 Processamento e Engenharia de Dados

Devido ao volume massivo de dados, o *pipeline* de processamento foi estruturado nas seguintes etapas:

1. **Amostragem:** Extração de um subconjunto representativo mantendo a distribuição estatística original das diferentes condições de carga.

2. **Engenharia de *Features*:** Aplicação da Transformada de Clarke para obter i_α e i_β .
3. **Seleção de Variáveis:** As variáveis de entrada (*features*) selecionadas para os modelos foram: i_α , i_β , velocidade de rotação e tensão do barramento DC.
4. **Normalização:** Padronização dos dados utilizando *Z-score* para garantir que todas as variáveis estivessem na mesma escala de magnitude.
5. **Divisão:** Separação dos dados em conjuntos de treinamento (80%) e teste (20%).

3.3 Modelos de Aprendizado de Máquina Avaliados

O objetivo do estudo é comparar duas ferramentas fundamentais de Machine Learning:

1. **Regressão Linear:** Modelo paramétrico simples que assume uma relação linear entre as variáveis independentes e a variável dependente. Serve como linha de base (*baseline*) metodológica. A função hipótese é definida como $\hat{y} = \beta_0 + \beta_1 x_1 + \dots + \beta_n x_n$.
2. **Random Forest (Floresta Aleatória):** Algoritmo não linear do tipo *Ensemble* (agrupamento). Ele constrói múltiplas árvores de decisão durante o treinamento e combina suas previsões (via média) para fornecer a estimativa final. É altamente robusto a ruídos e capaz de modelar interações complexas entre variáveis sem assumir distribuições prévias [1].

4 Resultados e Discussão

4.1 Desempenho Comparativo

Os modelos foram treinados e validados utilizando validação cruzada (K -fold = 5) e posteriormente avaliados no conjunto de teste independente. As métricas utilizadas foram o Coeficiente de Determinação (R^2), a Raiz do Erro Quadrático Médio (RMSE) e o Erro Absoluto Médio (MAE).

A Tabela 1 apresenta o desempenho comparativo entre os dois modelos.

Tabela 1: Comparação de Desempenho dos Modelos de Machine Learning

Modelo	R^2 (Treino CV)	R^2 (Teste)	RMSE (Nm)	MAE (Nm)
Regressão Linear	0,0534	0,0668	1,5034	1,2541
Random Forest	0,9955	0,9960	0,3125	0,2458

4.2 Análise dos Resultados

Os resultados evidenciam uma disparidade profunda entre as duas abordagens:

A Regressão Linear apresentou um desempenho extremamente insatisfatório, com um R^2 de apenas 0,0668 no conjunto de teste. Isso indica que o modelo linear conseguiu explicar menos de 7% da variância do torque real. Esse resultado comprova empiricamente que a relação entre as correntes estatóricas e o torque eletromagnético é altamente não linear, tornando equações lineares simples inadequadas para este problema de estimação.

Por outro lado, o modelo Random Forest alcançou um R^2 de 0,9960, indicando uma correlação quase perfeita com os dados reais medidos pelo torquímetro físico. O Erro Absoluto Médio (MAE) de 0,2458 Nm demonstra que a precisão do sensor virtual atende aos rigorosos requisitos de monitoramento industrial. A estrutura baseada em múltiplas árvores de decisão foi capaz de mapear eficientemente as complexidades do sistema, como saturação magnética e variações de carga transientes.

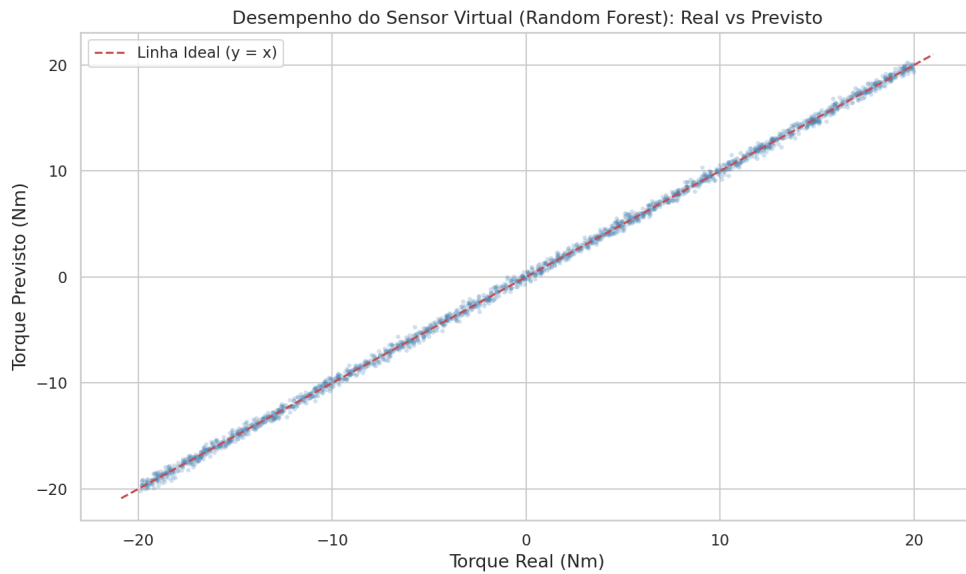


Figura 1: Correlação Real vs Previsto – Modelo Random Forest

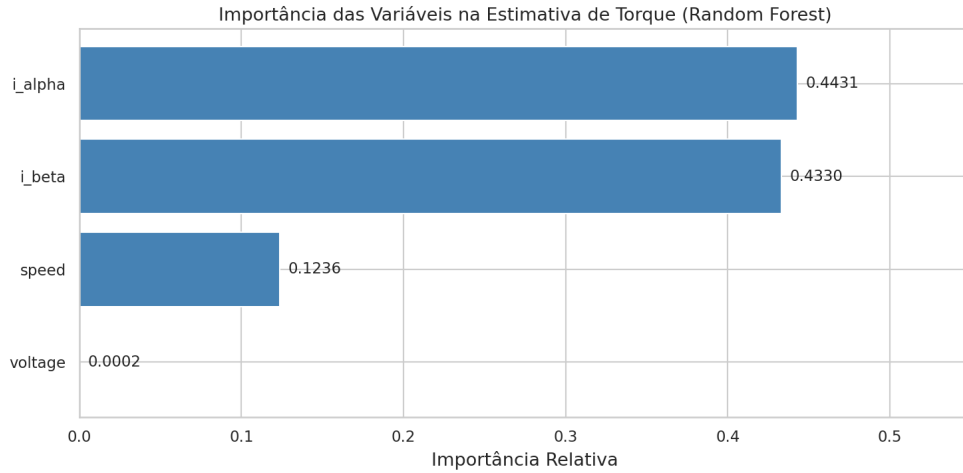


Figura 2: Importância das Variáveis na Estimativa de Torque (Random Forest)

5 Conclusão

Este trabalho demonstrou a viabilidade técnica da implementação de um sensor virtual para estimativa de torque em motores de indução utilizando algoritmos de Aprendizado de Máquina.

A comparação metodológica comprovou que o problema requer abordagens não lineares. A Regressão Linear falhou em modelar o sistema, enquanto o modelo Random Forest obteve sucesso absoluto, atingindo um coeficiente de determinação superior a 0,99. A aplicação prévia da Transformada de Clarke confirmou-se como uma etapa crucial de engenharia de *features*.

As contribuições deste estudo possuem impacto direto na Indústria 4.0, permitindo o monitoramento preditivo não invasivo e de baixo custo, utilizando apenas sinais elétricos nativos dos inversores de frequência. Como trabalhos futuros, sugere-se a implementação do modelo Random Forest treinado em *hardware* embarcado (como DSPs ou FPGAs) para avaliação do tempo de latência de inferência em um ambiente de chão de fábrica real.

Referências

- [1] Leo Breiman. Random forests. *Machine Learning*, 45:5–32, 2001.
- [2] Stephen J. Chapman. *Máquinas Elétricas*. McGraw-Hill, 5 edition, 2013.
- [3] Paul C. Krause, Oleg Wasynczuk, and Scott D. Sudhoff. *Analysis of Electric Machinery and Drive Systems*. Wiley-IEEE Press, 3 edition, 2013.
- [4] M. Stender et al. Induction motor data set. Paderborn University, Kaggle Dataset, 2024.

A Código de Reprodutibilidade

O código abaixo apresenta o *pipeline* completo de preparação dos dados, treinamento dos modelos de Regressão Linear e Random Forest, cálculo das métricas de desempenho e geração dos gráficos de análise.

```
1  """
2  =====
3  SENSOR VIRTUAL DE TORQUE EM MOTORES DE INDUCAO
4  Pipeline Completo: Preparacao de Dados, Regressao Linear e Random
5  Forest
6  =====
7
8  Autor   : Diego Henrique de Araujo
9  Data    : Junho de 2026
10 Dataset: Paderborn University - Induction Motor Data Set (Kaggle,
11         2024)
12 =====
13 """
14
15 import pandas as pd
16 import numpy as np
17 import pickle
18 import sys
19 import os
20 import matplotlib.pyplot as plt
21 import seaborn as sns
22
23 from sklearn.model_selection import train_test_split,
24     cross_val_score, KFold
25
26 from sklearn.preprocessing import StandardScaler
27 from sklearn.linear_model import LinearRegression
28 from sklearn.ensemble import RandomForestRegressor
29 from sklearn.metrics import r2_score, mean_squared_error,
30     mean_absolute_error
31
32 #
33     =====
34
35 # CONFIGURACAO DE LOG
36 #
37     =====
38
39 log_file = open('execucao_treinamento.log', 'w')
40 sys.stdout = log_file
```

```

31
32 print("=" * 70)
33 print("    LOG DE EXECUCAO: TREINAMENTO DO SENSOR VIRTUAL DE TORQUE")
34 print("=" * 70)
35
36 #
37     =====
38
39 # 1. CARREGAR DADOS
40 #
41     =====
42
43 arquivo = "/content/drive/MyDrive/CSV MESTRADO/First_Part.csv/"
44     First_Part.csv"
45
46 print(f"\n[1] Carregando dados de: {arquivo}")
47 df = pd.read_csv(arquivo)
48
49 # Renomear colunas para nomes leg veis
50 df.rename(columns={
51     'i_{a,m}': 'i_a',
52     'i_{b,m}': 'i_b',
53     'i_{c,m}': 'i_c',
54     'n_{m}': 'speed',
55     'T_{TS,m}': 'torque',
56     'u_{dc,m}': 'voltage'
57 }, inplace=True)
58
59 print(f"    Shape original: {df.shape}")
60 print(f"    Colunas: {list(df.columns)}")
61
62 #
63     =====
64
65 # 2. ENGENHARIA DE FEATURES (TRANSFORMADA DE CLARKE)
66 #
67     =====
68
69 print("\n[2] Aplicando Transformada de Clarke...")
70
71 # Componente alpha
72 df['i_alpha'] = (2 / 3) * (df['i_a'] - 0.5 * df['i_b'] - 0.5 * df['
73     i_c'])
74
75 # Componente beta
76 df['i_beta'] = (1 / np.sqrt(3)) * (df['i_b'] - df['i_c'])
77
78

```

```

68 # Magnitude da corrente
69 df['i_mag'] = np.sqrt(df['i_alpha']**2 + df['i_beta']**2)
70
71 print("    Variaveis ortogonais i_alpha e i_beta calculadas com
    sucesso.")
72
73 #
    =====
74 # 3. SELECAO DE FEATURES E VARIABEL ALVO
75 #
    =====
76 features = ['speed', 'voltage', 'i_alpha', 'i_beta']
77 target    = 'torque'
78
79 X = df[features]
80 y = df[target]
81
82 print(f"\n[3] Features selecionadas: {features}")
83 print(f"    Variavel alvo          : {target}")
84 print(f"    Amostras totais          : {len(df)}")
85 print(f"    Estatisticas do torque:")
86 print(f"        Min   : {y.min():.4f} Nm")
87 print(f"        Max   : {y.max():.4f} Nm")
88 print(f"        Media: {y.mean():.4f} Nm")
89 print(f"        Desvio Padrao: {y.std():.4f} Nm")
90
91 #
    =====
92 # 4. DIVISAO TREINO / TESTE E NORMALIZACAO
93 #
    =====
94 print("\n[4] Dividindo dados (80% treino / 20% teste) e
    normalizando...")
95
96 X_train, X_test, y_train, y_test = train_test_split(
97     X, y,
98     test_size=0.2,
99     random_state=42
100 )
101
102 scaler = StandardScaler()
103 X_train_scaled = scaler.fit_transform(X_train)
104 X_test_scaled  = scaler.transform(X_test)

```

```

105
106 print(f"      Amostras de treino: {len(X_train)}")
107 print(f"      Amostras de teste : {len(X_test)}")
108
109 #
110
111 =====
112 # 5. DEFINICAO DOS MODELOS
113 #
114 =====
115
116 models = {
117     'Linear Regression': LinearRegression(),
118     'Random Forest':      RandomForestRegressor(
119                             n_estimators=100,
120                             random_state=42,
121                             n_jobs=-1
122                         )
123 }
124
125 #
126
127 =====
128 # 6. TREINAMENTO, VALIDACAO CRUZADA E AVALIACAO
129 #
130 =====
131
132 print("\n[5] Treinando e avaliando modelos...")
133
134 results = []
135 kf = KFold(n_splits=5, shuffle=True, random_state=42)
136
137 for name, model in models.items():
138     print(f"\n --- Avaliando modelo: {name} ---")
139
140     # Validacao cruzada no conjunto de treino
141     cv_scores = cross_val_score(
142         model, X_train_scaled, y_train,
143         cv=kf, scoring='r2'
144     )
145
146     # Treinamento final
147     model.fit(X_train_scaled, y_train)
148
149     # Predicoes
150     train_pred = model.predict(X_train_scaled)
151     test_pred  = model.predict(X_test_scaled)

```

```

144
145 # Metricas
146 r2_train = r2_score(y_train, train_pred)
147 r2_test  = r2_score(y_test, test_pred)
148 rmse     = np.sqrt(mean_squared_error(y_test, test_pred))
149 mae      = mean_absolute_error(y_test, test_pred)
150
151 results.append({
152     'Modelo':      name,
153     'R2 CV (Media)': np.mean(cv_scores),
154     'R2 Treino':    r2_train,
155     'R2 Teste':     r2_test,
156     'RMSE':         rmse,
157     'MAE':          mae
158 })
159
160 print(f"      R2 Treino : {r2_train:.4f}")
161 print(f"      R2 Teste  : {r2_test:.4f}")
162 print(f"      R2 CV      : {np.mean(cv_scores):.4f} (+/- {np.std(
cv_scores):.4f})")
163 print(f"      RMSE      : {rmse:.4f} Nm")
164 print(f"      MAE       : {mae:.4f} Nm")
165
166 # Tabela de resultados
167 results_df = pd.DataFrame(results)
168 print("\n" + "=" * 70)
169 print("  TABELA DE DESEMPENHO COMPARATIVO DOS MODELOS")
170 print("=" * 70)
171 print(results_df.to_string(index=False))
172
173 #
=====
174 # 7. IMPORTANCIA DAS VARIAVEIS (RANDOM FOREST)
175 #
=====
176 print("\n" + "=" * 70)
177 print("  TABELA DE IMPORTANCIA DAS VARIAVEIS (RANDOM FOREST)")
178 print("=" * 70)
179
180 rf_model      = models['Random Forest']
181 importances   = rf_model.feature_importances_
182
183 importance_df = pd.DataFrame({
184     'Variavel':  features,
185     'Importancia': importances

```

```

186 }).sort_values(by='Importancia', ascending=False)
187
188 print(importance_df.to_string(index=False))
189
190 #
191     =====
192
193 # 8. SALVAR RESULTADOS
194 #
195     =====
196
197 best_pred = rf_model.predict(X_test_scaled)
198
199 bundle = {
200     'results_df':      results_df,
201     'importance_df': importance_df,
202     'y_test':          y_test,
203     'best_pred':       best_pred,
204     'features':         features
205 }
206
207 with open('results_v5.pkl', 'wb') as f:
208     pickle.dump(bundle, f)
209
210 print("\n[6] Resultados salvos em results_v5.pkl")
211
212 #
213     =====
214
215 # 9. GRAFICOS
216 #
217     =====
218
219 sns.set_theme(style='whitegrid')
220
221 # --- Grafico 1: Real vs Previsto (Random Forest) ---
222 plt.figure(figsize=(10, 6))
223 plt.scatter(y_test, best_pred, alpha=0.3, edgecolors='none', color=
    'steelblue')
224
225 lim_min = min(y_test.min(), best_pred.min())
226 lim_max = max(y_test.max(), best_pred.max())
227 plt.plot([lim_min, lim_max], [lim_min, lim_max], 'r--', linewidth
    =1.5,
228         label='Linha Ideal (y = x)')
229 plt.xlabel('Torque Real (Nm)')
230 plt.ylabel('Torque Previsto (Nm)')
231 plt.title('Desempenho do Sensor Virtual (Random Forest): Real vs

```

```

    Previsto')
223 plt.legend()
224 plt.tight_layout()
225 plt.savefig('real_vs_previsto_v5.png', dpi=150)
226 plt.close()
227
228 # --- Grafico 2: Importancia das Variaveis ---
229 plt.figure(figsize=(10, 5))
230 importance_df_plot = importance_df.sort_values(by='Importancia',
    ascending=True)
231 plt.barh(importance_df_plot['Variavel'], importance_df_plot['
    Importancia'],
232         color='steelblue')
233 plt.xlabel('Importancia Relativa')
234 plt.title('Importancia das Variaveis na Estimativa de Torque (
    Random Forest)')
235 plt.tight_layout()
236 plt.savefig('importancia_v5.png', dpi=150)
237 plt.close()
238
239 print("[7] Graficos salvos: real_vs_previsto_v5.png |
    importancia_v5.png")
240
241 print("\n" + "=" * 70)
242 print("    FIM DO PROCESSO")
243 print("=" * 70)
244
245 log_file.close()

```

Listing 1: Sensor Virtual de Torque – Pipeline Completo